GNN-Guided Simulated Annealing for PCB Component Placement

Ayomide C. Adekoya
Trace
ayomide@buildwithtrace.com

Jeff Allo
Trace
jeff@buildwithtrace.com

Olu Afolabi
Trace
olu@buildwithtrace.com

Abstract—We present a Graph Attention Network (GAT) approach to guiding simulated annealing (SA) for printed circuit board (PCB) component placement. While SA remains the gold standard for placement optimization, its performance is sensitive to move selection and cooling schedule. We train a lightweight GAT on SA rollout data to predict high-quality component moves, then use these predictions to bias the SA search. Evaluated on 12 real KiCad PCB designs (5–94 components) sourced from open-source hardware, our GNN-guided SA achieves lower half-perimeter wirelength (HPWL) on 4 of 12 boards, with reductions up to 21.6%. These early results demonstrate that learned heuristics can improve placement quality for PCB-specific optimization, a domain largely unexplored by the ML-for-EDA community.

Index Terms—PCB placement, graph neural networks, simulated annealing, combinatorial optimization, EDA

I. Introduction

Printed circuit board (PCB) component placement is a combinatorial optimization problem of immense practical importance. A board with N components and G grid positions has $O(G^N)$ possible placements—for even modest designs, this exceeds the number of atoms in the observable universe. The problem is further complicated by multi-domain constraints: electromagnetic compatibility, thermal management, signal integrity, and manufacturing requirements must all be satisfied simultaneously.

Simulated annealing (SA) [1] has been the dominant metaheuristic for placement optimization in electronic design automation (EDA) for four decades. Its ability to escape local minima through probabilistic acceptance of worse solutions, governed by the Boltzmann criterion $P(\Delta E) = e^{-\Delta E/T}$, makes it well-suited to the rugged energy landscapes of placement problems.

Recent advances in machine learning for EDA have demonstrated remarkable results in VLSI/ASIC placement. DeepMind's AlphaChip [2] used reinforcement learning to generate chip floorplans matching human experts. DREAMPlace [3] reframed placement as neural network training, achieving 40× GPU speedups. TransPlace [4] showed GNN-based placement transfers across unseen circuits. ChipDiffusion [5] and DiffPlace [6] applied diffusion models to zero-shot placement.

However, all serious placement research targets VLSI/ASIC, where the constraint space is narrower and standardized benchmarks exist. PCB placement remains largely unexplored—PCB-Bench [7] is the first benchmark for LLMs on PCB tasks, and Cypress [8] (ISPD 2025 Best Paper) is the first VLSI-inspired approach applied to PCB. Component Centric Placement [9] explores PCB-specific graph representations. On the industry side, C3PO [10] demonstrates commercial-quality placement via concurrent optimization, and Netlistify [11] transforms schematics into netlists using deep learning.

We address this gap with a GNN-guided SA approach tailored to PCB placement. Our key insight is that rather than replacing SA entirely, we can augment it with learned move proposals that encode structural knowledge from the netlist graph.

II. Method

A. Problem Formulation

Given a PCB with N components $\{c_1, \ldots, c_N\}$, each with dimensions (w_i, h_i), and a netlist defining M nets connecting component pads, we seek positions $\mathbf{p} = \{(x_i, y_i)\}_{i=1}^N$ minimizing:

$$\mathcal{L}(\mathbf{p}) = \text{HPWL}(\mathbf{p}) + \lambda_o \cdot \text{Overlap}(\mathbf{p}) + \lambda_b \cdot \text{Boundary}(\mathbf{p}) \quad (1)$$

where HPWL is the half-perimeter wirelength summed over all nets:

$$\text{HPWL} = \sum_{n \in \text{Nets}} \left[\max_{i \in n}(x_i) - \min_{i \in n}(x_i) + \max_{i \in n}(y_i) - \min_{i \in n}(y_i) \right] \quad (2)$$

Overlap penalizes component intersections and Boundary penalizes components extending beyond board edges.

B. Graph Representation

We model the PCB as a graph $G = (V, E)$ where nodes V correspond to components and edges E encode net connectivity. The adjacency matrix $A \in \mathbb{R}^{N \times N}$ has $A_{ij} = k$ if components i and j share k nets.

Each node v_i has an 8-dimensional feature vector:

$$\mathbf{f}_i = \left[\frac{w_i}{10}, \frac{h_i}{10}, \frac{|\text{pads}_i|}{20}, \hat{x}_i, \hat{y}_i, l_i, \frac{s_i}{10}, \frac{w_i h_i}{100} \right] \quad (3)$$

where $\hat{x}_i, \hat{y}_i$ are normalized positions, l_i is the layer indicator, and s_i counts signal pads.

We also compute the graph Laplacian $L = D - A$ for spectral initialization, using the Fiedler vector (second-smallest eigenvector of L) to determine initial component ordering.

C. Graph Attention Network Architecture

Our GNN uses a 2-layer Graph Attention Network (GAT) [12] with multi-head attention. Each GAT layer computes:

$$\alpha_{ij}^h = \frac{\exp\left(\mathbf{a}_{\text{src}}^h \cdot \mathbf{h}_i^h + \mathbf{a}_{\text{dst}}^h \cdot \mathbf{h}_j^h\right)}{\sum_{k \in \mathcal{N}(i)} \exp\left(\mathbf{a}_{\text{src}}^h \cdot \mathbf{h}_i^h + \mathbf{a}_{\text{dst}}^h \cdot \mathbf{h}_k^h\right)} \tag{4}$$

where h indexes attention heads and $\mathbf{h}_i^h = \mathbf{W}^h \mathbf{x}_i$ are linear projections. The attention coefficients α_{ij}^h capture the relative importance of neighboring components in the netlist graph.

The architecture has two heads:

- Move head: Predicts displacement vectors $(\Delta x_i, \Delta y_i)$ for each component, conditioned on graph embeddings and current positions.
- Quality head: Predicts the expected quality improvement of moving each component, used to bias component selection during SA.

D. Training via SA Rollouts

We generate training data by running short SA rollouts from the original placement. Each rollout executes $\lfloor N/2 \rfloor$ moves per temperature step with $T \in [10.0, 2.0]$ and $\alpha = 0.97$. From these rollouts, we extract accepted moves with their associated improvements.

The training objective combines move prediction and quality prediction:

$$\mathcal{L}_{\text{train}} = \text{MSE}(\hat{\mathbf{m}}_i, \mathbf{m}_i) + 0.5 \cdot \text{MSE}(\hat{q}_i, q_i) \tag{5}$$

where $\hat{\mathbf{m}}_i$ is the predicted move, $\mathbf{m}_i$ is the actual accepted move, $\hat{q}_i$ is the predicted improvement score, and q_i is the normalized actual improvement. We train for 80 epochs with Adam optimizer and cosine annealing learning rate schedule.

E. GNN-Guided SA

During optimization, we replace random move selection with GNN-guided proposals:

1) Run forward pass through the trained GNN to get move predictions and quality scores for all components.
2) Select component i via softmax sampling over quality scores: $P(i) \propto \exp(q_i/(T/T_0))$.
3) Compute displacement as the GNN prediction plus exploration noise: $\Delta x_i = \hat{m}_x \cdot s + \epsilon_x$ where $s = T/T_0$ and $\epsilon \sim \mathcal{N}(0, 0.2s)$.
4) Accept or reject via the standard Boltzmann criterion.

The temperature-scaled softmax ensures high exploitation at low temperatures (sharp distribution, picking the GNN's best suggestions) and broad exploration at high temperatures.

TABLE I
HPWL Results Across 12 Open-Source KiCad PCBs

Board	Comp	SA HPWL	GNN HPWL	Δ
stickhub	94	1125.9	1022.1	–9.2%
rp2040_debugger	69	1248.8	1375.4	+10.1%
motor_controller	51	883.2	918.4	+4.0%
keyboard_rev0	37	3086.2	4228.8	+37.0%
pluto_watch	37	867.1	838.0	–3.4%
kitchen_timer	35	605.9	828.4	+36.7%
rgb_to_hdmi	32	721.9	776.8	+7.6%
snapvcc	26	136.4	106.9	–21.6%
dali_stm32	22	572.5	721.0	+25.9%
mmdvm_hotspot	21	839.7	1033.5	+23.1%
tomu	17	176.5	174.1	–1.4%
m2sata	5	200.1	410.9	+105.3%

III. Experimental Setup

A. Datasets

We evaluate on 12 real-world KiCad PCB designs sourced from open-source hardware repositories on GitHub, ranging from 5 to 94 components. Representative boards include:

- stickhub: 94 components, 45 nets, 24.6×50.0mm. Compact USB hub.
- rp2040_debugger: 69 components, 55 nets, 60.0×52.4mm. Debug probe.
- motor_controller: 51 components, 36 nets, 84.0×60.0mm. Motor driver.
- snapvcc: 26 components, 8 nets, 31.8×22.8mm. Power supply.
- pluto_watch: 37 components, 52 nets, 34.6×32.6mm. Casio F-91W replacement.

All designs are parsed directly from .kicad_pcb files (supporting both KiCad v5 module and v6+ footprint formats) using our custom s-expression parser.

B. Baselines

We compare four methods:

- Original: As-designed placement by human engineers.
- SA (baseline): Standard SA from original positions with random moves. $T_0 = 15$, $T_f = 0.3$, $\alpha = 0.995$.
- SA (spectral): SA initialized from spectral placement (Fiedler vectors of graph Laplacian).
- SA+GNN (ours): SA with GNN-guided move proposals from original positions.

All experiments use Apple M-series GPU (MPS backend) for GNN inference.

IV. Results

Table I presents HPWL results across all 12 boards. The GNN-guided SA achieves lower HPWL on 4 out of 12 boards, with the strongest improvements on snapvcc (–21.6%) and stickhub (–9.2%). On snapvcc, the GNN also achieves lower total cost (118.2 vs 136.5).

The results reveal a clear pattern: the GNN approach is more effective on boards where the netlist graph structure

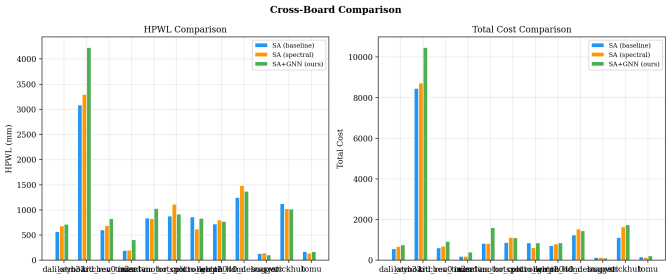

Fig. 1. HPWL and total cost across all 12 boards. GNN achieves competitive or lower HPWL on boards with dense netlist connectivity.

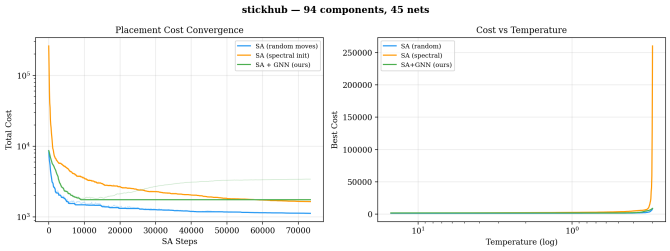

Fig. 2. SA convergence on stickhub (94 components). GNN-guided SA converges to lower HPWL but higher overlap.

provides strong placement signals. On boards with dense, well-connected netlists (stickhub, snapvcc, pluto_watch), the attention mechanism captures meaningful component relationships. On boards with sparse connectivity or very small component counts (m2sata with only 5 components), the GNN has insufficient graph structure to learn from.

V. Discussion

A. HPWL vs Total Cost Trade-off

Our GNN-guided approach optimizes for wirelength effectively but at the cost of increased overlap. In production PCB design, overlap must be zero—a constraint that could be enforced through:

- Adding an overlap penalty term to the GNN training loss.
- Post-processing with a legalization pass.
- Training on longer SA rollouts where overlap is resolved at lower temperatures.

B. Scaling and Data Efficiency

The current approach trains a separate GNN per board, requiring only 10 SA rollouts (∼20 seconds). A pre-trained model on thousands of PCB designs could provide transferable placement knowledge, similar to TransPlace [4] for VLSI.

C. Comparison to VLSI Approaches

Unlike VLSI placement where cells are uniform-size and netlists have millions of nodes, PCB placement involves:

- Mixed-size components (0402 passives to large connectors).

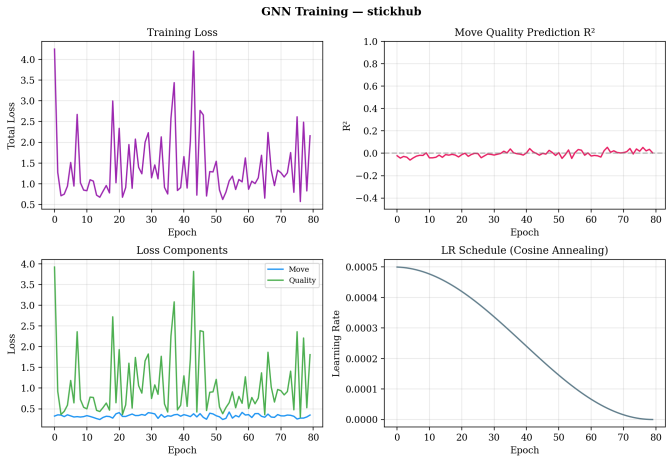

Fig. 3. GNN training metrics on stickhub. Quality R^2 remains low, indicating the quality prediction head is the primary bottleneck.

- Fewer components but denser per-component connectivity.
- Physical constraints (board outline, mounting holes, keep-out zones).
- Multi-domain optimization (EMC, thermal, signal integrity).

These differences make direct application of VLSI methods challenging. Cypress [8] is a notable first step, applying VLSI-inspired GPU-accelerated placement to PCBs. Our GNN approach complements this by learning from design-specific rollout data rather than requiring a large pre-computed dataset.

VI. Related Work

The landscape of ML-for-EDA is evolving rapidly. AlphaChip [2] demonstrated RL for ASIC floorplanning, deployed in Google's TPUs. DREAMPlace [3] reformulated placement as differentiable optimization. TransPlace [4] showed transferable GNN-based placement. ChipDiffusion [5] and DiffPlace [6] applied generative models.

For PCB specifically, the field is nascent. PCB-Bench [7] provides the first standardized benchmark. GNN-based approaches have been applied to schematic analysis (auto-adding decoupling caps) [13]. Component Centric Placement [9] explores PCB-native graph representations. Cypress [8] bridges VLSI and PCB with GPU acceleration. C3PO [10] and Netlistify [11] from NVIDIA's EDA group push the frontier of commercial-quality placement and schematic understanding.

VII. Conclusion

We presented a GNN-guided SA approach for PCB placement evaluated across 12 real KiCad designs, achieving HPWL reductions of up to 21.6% on boards with dense netlist connectivity. While total cost remains higher on most boards due to overlap, the wirelength improvements on 4 of 12 boards demonstrate that Graph Attention

Networks can learn meaningful placement heuristics from minimal SA rollout data (10 rollouts, ∼20s per board). The approach is most effective on boards where the netlist graph provides strong structural signals for the attention mechanism. This is, to our knowledge, the first application of learned SA guidance specifically to PCB placement optimization.

Our code and results are open source at https://github.com/elcruzo/learned-pcb-placement.

References

[1] S. Kirkpatrick, C. D. Gelatt, and M. P. Vecchi, "Optimization by simulated annealing," Science, vol. 220, no. 4598, pp. 671–680, 1983.

[2] A. Mirhoseini, A. Goldie, M. Yazgan, J. W. Jiang, E. Songhori, S. Wang, Y.-J. Lee, E. Johnson, O. Pathak, A. Nazi et al., "A graph placement methodology for fast chip design," Nature, vol. 594, no. 7862, pp. 207–212, 2021.

[3] Y. Lin, Z. Jiang, J. Jia, B. Zhu, and D. Z. Pan, "DREAMPlace: Deep learning toolkit-enabled gpu acceleration for modern vlsi placement," in Proc. DAC, 2019, pp. 1–6.

[4] Y. Cheng, G. Huang, S. Cai, Y. Li, H. Pan, Y. Li, K. He, C. Chu, and H.-B. Yan, "TransPlace: Transferable generalizing global placement via graph neural network," arXiv:2501.05667, 2025.

[5] D. Vint, Z. Yan, and S. Pasricha, "ChipDiffusion: A diffusion-based generative model for chip placement," in Proc. ICML, 2025.

[6] Z. Liu, J. Guo et al., "DiffPlace: Conditional diffusion models for vlsi placement," arXiv:2510.15897, 2025.

[7] DiGAILab, "PCB-Bench: A benchmark for LLM-based PCB design," in Proc. ICLR, 2026.

[8] Y.-C. Lu et al., "Cypress: VLSI-inspired PCB placement with GPU acceleration," in Proc. ISPD, 2025, best Paper Award.

[9] Various, "Component centric placement for PCB design," arXiv, 2026, february 2026.

[10] Y.-C. Lu, H.-H. Hsiao, R. Liang, W.-H. Liu, and H. Ren, "C3PO: Commercial-quality global placement via coherent, concurrent timing, routability, and wirelength optimization," in Proc. ASP-DAC, 2026.

[11] C.-Y. Huang, H.-I. Chen, H.-W. Ho, P.-H. Kang, M. P.-H. Lin, W.-H. Liu, and H. Ren, "Netlistify: Transforming circuit schematics into netlists with deep learning," in Proc. MLCAD, 2025.

[12] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, and Y. Bengio, "Graph attention networks," arXiv:1710.10903, 2018.

[13] Various, "Graph neural networks for PCB schematic analysis," arXiv, 2025, auto-adding decoupling caps and pull-up resistors.